

Research Agent

Project report: what has been built, how it works, and what it costs

1 October 2026

Item	Status
Analysis of ResearchRadar (IDEASHACK winner) architecture	Done
Analysis of the OpenAlex Works API and its JSON	Done
Keyword search UI with paper cards, insights, AI brief	Done and tested
"Describe your problem" matching with relevance scores	Done and tested
LLM understanding (spelling fix, quotes, term mapping, hypotheses)	Done and tested with Groq
Cost analysis per step, live cost tracking in the UI	Done

1. Executive summary

The goal was to understand how **ResearchRadar** (researchradar.dev) works, and to build a working prototype of the same idea, named **Research Agent**: a user describes what they are working on and the problem they face, and the system returns related research papers with relevance scores and an explanation of how it found them.

- The data source is **OpenAlex**, a free open index of about 250 million scholarly works.
- A **Node.js backend** wraps OpenAlex: it cleans the data, rebuilds abstracts, caches responses and records the cost of every call.
- A **single-page web frontend** offers two modes: *Describe your problem* (AI matching) and *Keyword search*.
- An **LLM** (Groq, model openai/gpt-oss-120b; Claude supported as an alternative) understands the problem and re-ranks papers by meaning.
- Every score is **explainable**: the UI shows the score breakdown, a reason, and an evidence quote that is checked against the abstract.
- A full problem-to-results run costs about **\$0.008** and takes about 13 seconds.

2. How ResearchRadar works (analysis)

ResearchRadar does not publish its code, so this is a reconstruction from its public site. Its claim of "240M+ papers and 123k+ organizations, indexed nightly" matches the size of OpenAlex, so OpenAlex is very likely its data source.

Feature on their site	Likely implementation
Upload a paper/proposal; AI "interviews" it and keeps quotes	LLM extracts problems/topics plus supporting quotes from the document
Nightly sweep of new papers, not the full firehose	Cron job pulls OpenAlex works created yesterday, stores them locally
"87% match" alerts	Embedding similarity between user topics and new papers, thresholded
Competitor lab and shared-author alerts	Join on OpenAlex author and institution IDs
Ignition (new topic pairs), warming topics	Topic co-occurrence and counts per year over time
TRL 1-9 score with adversarial check	LLM scores the abstract with quotes; a second LLM call checks the claim
Browser extension, MCP server, weekly digest	Lookup by DOI against their own database; email job

Key design idea: they ingest once per night and do cheap structured matching (IDs, topics) on their own database, then run expensive LLM steps only on the few candidates that pass. That is how they can offer a plan with only a \$1/month AI allowance.

3. The OpenAlex API

Example request analysed:

```
GET https://api.openalex.org/works?search=microplastics
&filter=from_publication_date:2024-01-01&sort=cited_by_count:desc
```

It returned 71,621 works; the top one was *Twenty years of microplastic pollution research* (Science, 2024, 1,464 citations). Note that search= is rewritten to full-text search; the demo uses title_and_abstract.search for more precise results.

Fields used in the frontend

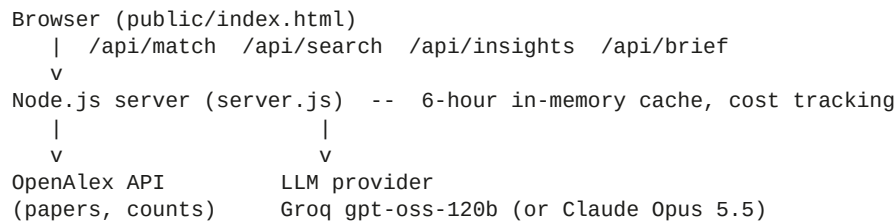
UI element	OpenAlex JSON field
Title, date	display_name, publication_date
Journal	primary_location.source.display_name
Authors, institution, country	authorships[].author.display_name, institutions[].display_name, country_code
Citations, trend sparkline	cited_by_count, counts_by_year[]
Top 1% / Top 10% badge, Impact score	citation_normalized_percentile
Open access badge, PDF link	open_access.is_oa, oa_status, best_oa_location.pdf_url
Field, keywords	primary_topic.field, keywords[]
Abstract	abstract_inverted_index (rebuilt into text)
Keyword relevance	relevance_score (returned when searching)

Rebuilding the abstract

OpenAlex stores abstracts as an inverted index (word -> positions) for licensing reasons. The backend turns it back into text:

```
function rebuildAbstract(inv) {  
  const words = [];  
  for (const [word, positions] of Object.entries(inv))  
    for (const p of positions) words[p] = word;  
  return words.join(" ");  
}
```

4. What was built: architecture



File	Role
server.js	HTTP server; OpenAlex proxy; normalisation; caching; LLM calls; problem-matching pipeline
public/index.html	Whole frontend: two tabs, paper cards, score bars, insights charts, pipeline trace, cost panel
.env (git-ignored)	GROQ_API_KEY, optional OPENALEX_API_KEY / ANTHROPIC_API_KEY
package.json	One dependency (@anthropic-ai/sdk, only used if Claude is chosen); npm start loads .env

API routes

Route	What it does	OpenAlex cost
POST /api/match	Problem text -> scored related studies + step-by-step trace	4-6 × \$0.001
GET /api/search	Keyword search, 20 papers per page, cursor paging	\$0.001
GET /api/insights	Papers per year, top fields, institutions, countries (group_by)	4 × \$0.0001
POST /api/brief	LLM summary of the top 10 search results	none

5. Problem matching: how it is done

This is the core feature. The pipeline has five steps; each one is timed, costed and shown to the user in the "How these were found" panel.

Step	What happens	Why
1. Understand (LLM)	Fixes spelling; copies the user's exact words for each aspect (quote) and maps them to scientific terms and synonyms; proposes hypotheses that could connect the aspects; picks a research domain; writes 4-6 short queries that COMBINE aspects.	Literal typos ("hairloss") match nothing; single-aspect queries return papers about only one part of the problem.
Quote check (code)	Each quote must actually appear in the user's text.	Stops the LLM inventing aspects.
2. Retrieve	All queries run in parallel on OpenAlex title+abstract search, limited to the chosen domain (e.g. Health Sciences), 25 results each; results pooled and de-duplicated.	Domain filter removes off-field papers (e.g. agriculture for a medical question).
3. Pre-filter	Keyword score = $0.5 \times \text{OpenAlex relevance} + 0.25 \times \text{share of queries that found the paper} + 0.25 \times \text{concept coverage}$. Top 30 kept.	Free; keeps the LLM step small and cheap.
4. Re-rank (LLM)	The LLM reads 30 abstracts and scores 0-100 for the problem as a whole, gives a reason, lists which aspects it covers, and copies an exact evidence sentence.	Meaning, not word overlap. Multi-aspect papers rank higher.
Evidence check (code)	The evidence sentence is searched for in the real abstract. If it is not there, the score is cut by 25%.	Guards against invented justifications.
5. Combine	Final = 65% Meaning + 20% Keywords + 15% Impact (citation percentile).	Relevance first, with a small boost for well-cited work.

Structured LLM output

Both LLM calls use a strict JSON schema, so the response is always machine-readable. Example of the understanding schema (simplified):

```
{ "corrected_text": "...", "summary": "...",  
  "domain": "health | life | physical | social | any",  
  "key_phrases": [{ "quote": "hairloss", "term": "alopecia", "synonyms": [...] }],  
  "hypotheses": [{ "name": "Hyperthyroidism", "why": "..."}],  
  "queries":      [{ "q": "hyperthyroidism alopecia", "intent": "..."}] }
```

6. Test: before and after the LLM upgrade

Input typed by the user (with spelling mistakes):

i am facing hairloss and i have had severe weighloss and abnormal behavoiur

Before (keyword heuristic only): the typos were searched literally and each word was searched alone, so the results were irrelevant. In an earlier engineering test the same weakness ranked a SARS-CoV-2 wastewater paper at 53 for a microplastic-filtration problem.

After (Groq gpt-oss-120b):

User's words (quote)	Mapped research term	Synonyms
hairloss	alopecia	hair loss, telogen effluvium, androgenetic alopecia
severe weighloss	weight loss	cachexia, unintentional weight loss, wasting
abnormal behavoiur	behavioral abnormality	behavioral changes, neuropsychiatric symptoms

Hypotheses: hyperthyroidism; nutritional deficiency; chronic stress or depression. Domain: health. Queries: *hyperthyroidism alopecia, hyperthyroidism weight loss, nutritional deficiency alopecia, nutritional deficiency behavior, stress telogen effluvium.*

Score	Top results
68	Alopecia Universalis Associated with Hyperthyroidism ... A Case Report
68	Telogen Effluvium: A Review of the Literature
67	Hyperthyroidism (describes unintentional weight loss)
62	Chronological association between alopecia areata and autoimmune thyroid diseases (meta-analysis)
59	Alopecia and zinc deficiency in postbariatric surgery patients

124 candidates were pooled, 30 re-ranked, and 21 of 30 evidence quotes were verified in the abstracts. Because the domain is health, the UI shows a notice that these are research papers, not a diagnosis.

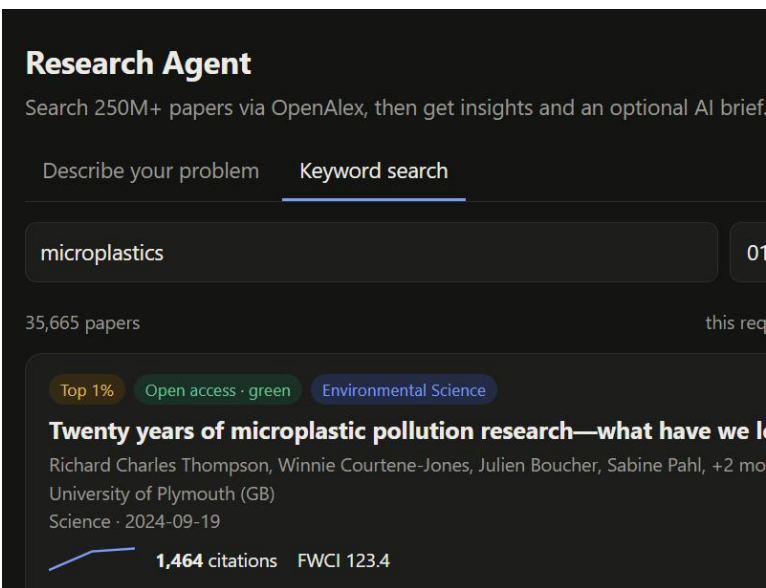
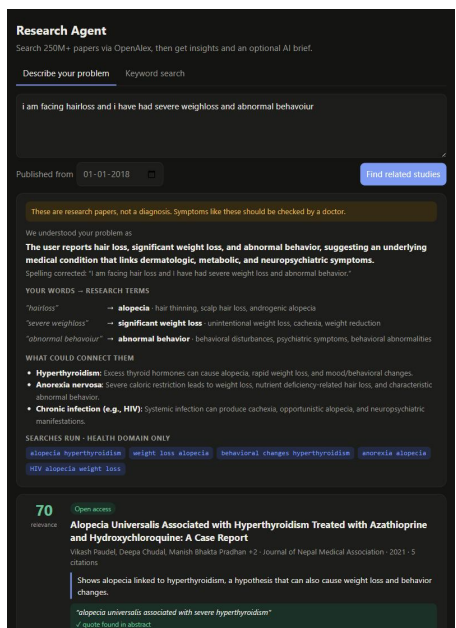


Figure 1. Problem matching on the same input (a later run; LLM wording and hypotheses vary slightly between runs).

Figure 2. Keyword search: paper cards, badges, sparkline, insights.

7. Cost analysis

OpenAlex prices were measured from the X-RateLimit-Cost-USD header of each response. Without an API key, OpenAlex gives \$0.10 of free usage per day.

OpenAlex call	Cost
Single paper lookup, autocomplete	Free
Filter-only list (topic ID, date, institution), group_by counts	\$0.0001
Text search (search= or title_and_abstract.search)	\$0.001 (10× more)

LLM prices (USD per million tokens, input / output): Groq gpt-oss-120b \$0.15 / \$0.60; gpt-oss-20b \$0.075 / \$0.30; Claude Haiku 4.5 \$1 / \$5; Sonnet 5.5 \$2 / \$10; Opus 5.5 \$4 / \$20. Claude's Batch API halves the price.

Measured cost of the hair-loss test run (Groq)

Step	Time	Tokens (in / out)	Cost
1. Understand	3.3 s	672 / 1,231	\$0.0008
2. Retrieve (5 OpenAlex searches)	2.5 s	-	\$0.0050
3. Pre-filter	0 s	-	\$0
4. Re-rank 30 abstracts	7.3 s	4,988 / 3,093	\$0.0026
5. Combine	0 s	-	\$0
Total	~13 s		\$0.0084

The same pipeline on Claude Opus 5.5 would cost roughly \$0.07-0.08 per question (estimate). With Groq, OpenAlex searches are now the largest cost item.

Cost per study at scale (estimates)

Configuration	Per question	1,000 questions
No LLM (keyword heuristic)	~\$0.004	~\$4
Groq gpt-oss-120b (current)	~\$0.008	~\$8
Claude Opus 5.5 for both LLM steps	~\$0.08	~\$80
ResearchRadar-style extras (TRL scoring of 10 papers)	+\$0.01-0.14	+\$10-140
Nightly watch per user topic (filter call + batch scoring)	~\$0.15 / month	

How to keep it cheap

- Use text search once, then cheap filters: nightly checks by topic ID cost \$0.0001 instead of \$0.001.
- Filter with free signals (citations, topic, institution) before paying for an LLM.
- Cache everything: OpenAlex only updates daily; repeat queries cost \$0.
- Use cheap or batched models for bulk scoring; keep the strongest model for the text the user reads.

8. Reading the badges

Top 1% / Top 10%: the paper's citations compared with papers from the same field and publication year (OpenAlex citation_normalized_percentile). Top 1% means more cited than 99% of comparable papers. It is a measure of influence, not of correctness or relevance; new papers rarely have it.

Open access type	Meaning
Gold	Published in a fully open journal
Diamond	Open journal that also charges authors nothing
Hybrid	Subscription journal; authors paid to make this paper free
Bronze	Free on the publisher's site but without an open license
Green	Journal version paywalled; free copy in a repository (university, preprint server)

9. Known limitations and next steps

- **Near-duplicate results:** e.g. three similar COVID-19 telogen effluvium papers. Next: cluster similar results and show one per cluster.
- **Occasional stretched reasoning** by the re-ranker. Next: an adversarial second LLM check on the top 5 (as ResearchRadar does for TRL).
- **Keyword retrieval only.** Next: store embeddings of abstracts (e.g. Postgres + pgvector) for true semantic search, as ResearchRadar likely does.
- **No persistence:** the cache is in memory. Next: Postgres for papers, saved problems ("watches"), and a nightly job with email digests.
- **Security:** the Groq key was shared in chat; it is stored only in a git-ignored .env file and should be rotated after testing.

10. How to run

```
cd Research-Agent
npm install
npm start          # reads .env, serves http://localhost:3000
```

If GROQ_API_KEY is set, Groq is used; otherwise ANTHROPIC_API_KEY (Claude); with neither, the demo falls back to the keyword heuristic.